Reduced-Order Modelling

Scientific Machine Learning and Simulation Intelligence

Reduced-Order Modelling

Reduced-order modelling in a nutshell

High/full order methods (FEM,FV,FD,SEM) to be accelerated using ROMs: $()^{\mathcal{N}}$

Reduced order model (ROM) which is the accelerator: $()_N$

Input parameters:

$$\mu \text{ (geometry, physical properties, etc.)}$$

Parametrised PDEs:

$$\mathcal{A}(u(\mu);\mu) = 0 \rightarrow \mathbf{A}^{\mathcal{N}}(\mu)\mathbf{u}^{\mathcal{N}}(\mu) = 0 \rightarrow \mathbf{A}_N(\mu)\mathbf{u}_N(\mu) = 0$$

Output:

$$s(\mu) \approx s^{\mathcal{N}}(\mu) \approx s_N(\mu)$$

Reduced-Order Modelling

Reduced-order modelling in a nutshell

Input parameters:

$$\mu \text{ (geometry, physical properties, etc.)}$$

Parametrised PDEs:

$$\mathcal{A}(u(\mu);\mu) = 0 \rightarrow \mathbf{A}^{\mathcal{N}}(\mu)\mathbf{u}^{\mathcal{N}}(\mu) = 0 \rightarrow \mathbf{A}_N(\mu)\mathbf{u}_N(\mu) = 0$$

Output:

$$s(\mu) \qquad \approx \qquad s^{\mathcal{N}}(\mu) \qquad \approx \qquad s_N(\mu)$$

Input-output evaluation:

$$\mu \qquad \rightarrow \qquad s^{\mathcal{N}}(\mu) \qquad \rightarrow \qquad s_N(\mu)$$

Reduced-Order Modelling

Reduced-order modelling in a nutshell

Input parameters:

$$\mu \text{ (geometry, physical properties, etc.)}$$

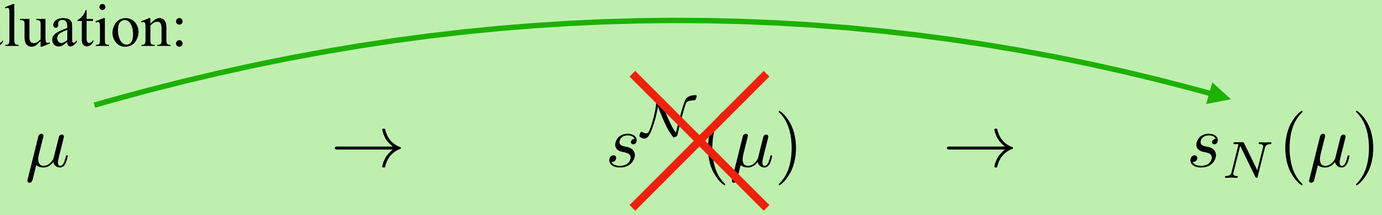
Parametrised PDEs:

$$\mathcal{A}(u(\mu);\mu) = 0 \rightarrow \mathbf{A}^{\mathcal{N}}(\mu)\mathbf{u}^{\mathcal{N}}(\mu) = 0 \rightarrow \mathbf{A}_N(\mu)\mathbf{u}_N(\mu) = 0$$

Output:

$$s(\mu) \qquad \approx \qquad s^{\mathcal{N}}(\mu) \qquad \approx \qquad s_N(\mu)$$

Input-output evaluation:



Reduced-Order Modelling

Reduced-order modelling in a nutshell

High/full order methods (FEM,FV,FD,SEM) to be accelerated using ROMs: $()^{\mathcal{N}}$

Reduced order model (ROM) which is the accelerator: $()_N$

offline

very expensive pre-processing (high/full order): basic calculation (done **once**)

online

extremely fast (reduced order): real-time input-output evaluation

$$\mu \quad \rightarrow \quad s_N(\mu)$$

thanks to an efficient assembly of problem operators

challenges: stability, error bounds, efficient parametrisation, sampling

Reduced-Order Modelling

FEM

If we go back to our problem, where everything is basically the same but now we have a different basis

$$-\kappa \int_0^1 \frac{\partial \zeta_j}{\partial x} \frac{\partial (\sum_{i=1}^{\mathcal{N}} u_i \zeta_i)}{\partial x} \, dx = \int_0^1 f(x,\mu)\zeta_j \, dx \qquad \forall j = 1, .., N$$

Simplifying you get

$$-\kappa \sum_{i=1}^{N} u_i \int_0^1 \frac{\partial \zeta_j}{\partial x} \frac{\partial \zeta_i}{\partial x} \, dx = \int_0^1 f(x,\mu)\zeta_j \, dx \qquad \forall j = 1, .., N$$

In matrix form you get

$$-\kappa \mathbf{K}^N \mathbf{u}^N = \mathbf{f}^N$$

Reduced-Order Modelling

We define $C^{POD} \in \mathbb{R}^{n_{train}} \times \mathbb{R}^{n_{train}}$ as:

$$C_{ij}^{POD} = \frac{1}{n_{train}} (u(\mu_{train}^i), u(\mu_{train}^j))_X, \quad 1 \leq i, j \leq n_{train}$$

Then we look at the eigenproblem

$$C^{POD} \zeta^k = \lambda^k \zeta^k$$

where we search for the eigenpairs $(\zeta^k \in \mathbb{R}^{n_{train}}, \lambda^k \in \mathbb{R}^{+,0})$, $1 \leq k \leq n_{train}$.
Arranging the eigenbalues in discending order

$$\lambda^1 \geq \lambda^2 \geq ... \geq \lambda^{n_{train}} \geq 0,$$

we can now identify a new basis in ζ_j

Moreover, we can define $N_{\max}$ as the smallest N such that

$$\epsilon_N^{POD} = \sqrt{\sum_{k=N+1}^{n_{train}} \lambda^k} \leq \epsilon_{\min}$$

The POD-RB spaces are $X_N^{POD} = \text{span}\{\zeta^n, 1 \leq n \leq N\}$, $1 \leq N \leq \mathcal{N}_{\max}$ with $(\zeta_n, \zeta_m)_X = \delta_{nm}, \quad 1 \leq n, m \leq n_{train}$

Reduced-Order Modelling

It can be proven that among all the possible basis of reduced dimensionality, the one provided by the POD is the optimal one that minimises the projection error

$$X_N^{\mathcal{N},POD} = \underset{X_N^{\mathcal{N}} \subset \text{span}\{u^{\mathcal{N}}(\mu)|\mu \in \Gamma_{train}\}}{\arg\inf} \|u^{\mathcal{N}} - \Pi_{X_N^{\mathcal{N}}} u^{\mathcal{N}}\|_{L^2(\Gamma_{train};X)},$$

The modes are ordered by importance:

$$\mathcal{C}^{\mathcal{N}} \zeta_i^{\mathcal{N}} = \lambda_i^{\mathcal{N}} \zeta_i^{\mathcal{N}}, \qquad \lambda_1^{\mathcal{N}} \geq \lambda_2^{\mathcal{N}} \geq \cdots \geq 0.$$

You then keep only the first modes:

$$X_N^{\mathcal{N},\text{POD}} = \text{span}\left\{\zeta_1^{\mathcal{N}}, \ldots, \zeta_N^{\mathcal{N}}\right\}.$$

You can then prove that the projection error is related to the eigenvalues of the neglected modes

$$\left\|u^{\mathcal{N}} - \Pi_{X_N^{\mathcal{N},\text{POD}}} u^{\mathcal{N}}\right\|^2_{L^2(\Gamma_{\text{train}};X)} = \sum_{i=N+1}^{\mathcal{N}} \lambda_i^{\mathcal{N}}.$$

Reduced-Order Modelling

FEM for advection diffusion

Intrusive ROM for the advection diffusion equation in 1D using the Finite Element Method:

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = \frac{1}{\mathrm{Pe}} \frac{\partial^2 u}{\partial x^2}$$

Weak formulation:

$$\int_0^1 \frac{\partial u}{\partial t} v \, dx + \int_0^1 \frac{\partial u}{\partial x} v \, dx = \int_0^1 \frac{1}{\mathrm{Pe}} \frac{\partial^2 u}{\partial x^2} v \, dx$$

Integration by parts:

$$\int_0^1 \frac{\partial u}{\partial t} v \, dx + \int_0^1 \frac{\partial u}{\partial x} v \, dx = -\frac{1}{\mathrm{Pe}} \int_0^1 \frac{\partial u}{\partial x} \frac{\partial v}{\partial x} \, dx$$

Reduced-Order Modelling

FEM for advection diffusion

We now introduce the FE approximation:

$$u(x) = \sum_{i=1}^{\mathcal{N}} u_i \phi_i(x)$$

Using this in addition with testing on $\phi_j(x)$ we get:

$$\int_0^1 \frac{\partial \sum_{i=1}^{\mathcal{N}} u_i \phi_i}{\partial t} \phi_j \, dx + \int_0^1 \frac{\partial \sum_{i=1}^{\mathcal{N}} u_i \phi_i}{\partial x} \phi_j \, dx = -\frac{1}{\mathrm{Pe}} \int_0^1 \frac{\partial \sum_{i=1}^{\mathcal{N}} u_i \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} \, dx$$

After some simplification you get:

$$\sum_{i=1}^{\mathcal{N}} \frac{\partial u_i}{\partial t} \int_0^1 \phi_i \phi_j \, dx + \sum_{i=1}^{\mathcal{N}} u_i \int_0^1 \frac{\partial \phi_i}{\partial x} \phi_j \, dx = -\frac{1}{\mathrm{Pe}} \sum_{i=1}^{\mathcal{N}} u_i \int_0^1 \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} \, dx$$

Reduced-Order Modelling

FEM for advection diffusion

$$\sum_{i=1}^{\mathcal{N}} \frac{\partial u_i}{\partial t} \int_0^1 \phi_i \phi_j \, dx + \sum_{i=1}^{\mathcal{N}} u_i \int_0^1 \frac{\partial \phi_i}{\partial x} \phi_j \, dx = -\frac{1}{\mathrm{Pe}} \sum_{i=1}^{\mathcal{N}} u_i \int_0^1 \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} \, dx$$

Reduced-Order Modelling

FEM for advection diffusion

$$\sum_{i=1}^{\mathcal{N}} \frac{\partial u_i}{\partial t} \int_0^1 \phi_i \phi_j \, dx + \sum_{i=1}^{\mathcal{N}} u_i \int_0^1 \frac{\partial \phi_i}{\partial x} \phi_j \, dx = -\frac{1}{\mathrm{Pe}} \sum_{i=1}^{\mathcal{N}} u_i \int_0^1 \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} \, dx$$

In matrix form we have:

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \mathbf{C}\mathbf{u} = -\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}$$

With

$$M_{ij} = \int_0^1 \phi_i \phi_j dx$$

$$C_{ij} = \int_0^1 \frac{\partial \phi_i}{\partial x} \phi_j dx$$

$$K_{ij} = \int_0^1 \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} dx$$

Reduced-Order Modelling

FEM for advection diffusion

In matrix form we have:

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \mathbf{C}\mathbf{u} = -\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}$$

Reduced-Order Modelling

FEM for advection diffusion

In matrix form we have:

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \mathbf{C}\mathbf{u} = -\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}$$

If we also add time integration we get our final model:

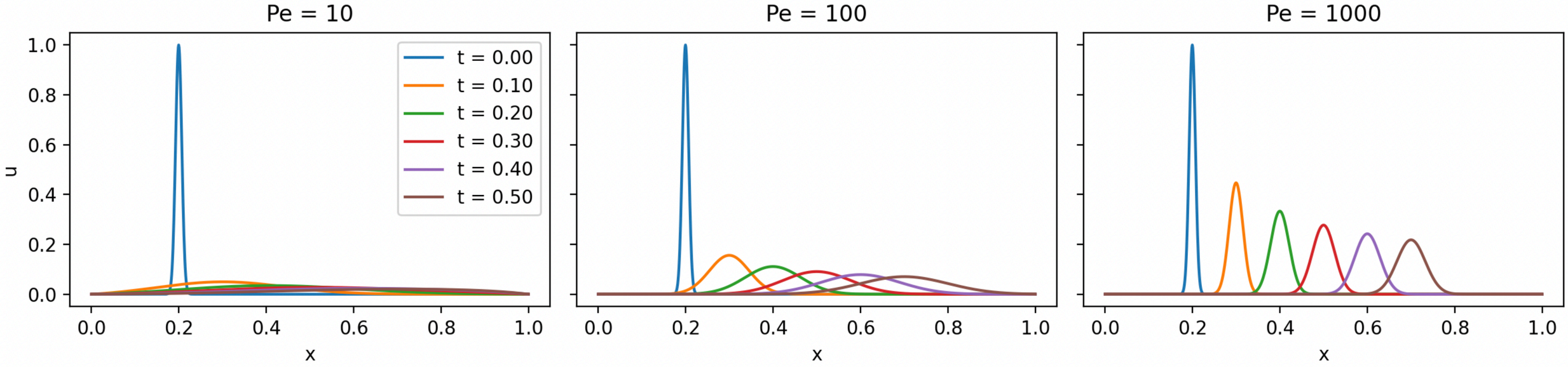
$$\mathbf{M}\frac{\mathbf{u}^{t+1} - \mathbf{u}^{t}}{\Delta t} + \mathbf{C}\mathbf{u}^{t} = -\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}^{t}$$

Reduced-Order Modelling

FEM for advection diffusion

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = \frac{1}{\mathrm{Pe}} \frac{\partial^2 u}{\partial x^2}$$

At the moment we will consider time to be our parameter: The objective of the ROM is to recover the same accuracy of the FE method in a much faster way involving significantly fewer total degrees of freedom



Reduced-Order Modelling

FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, number of elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, C
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$

```
# ---------------- parameters
n_el = 1000
T, dt = 0.5, 1e-6
save_every = 1000
plot_every = 1000
Pe_values = [10,100,1000]
    checks)¬
x0, w0 = 0.2, 0.01
# ------------------------
```

Reduced-Order Modelling

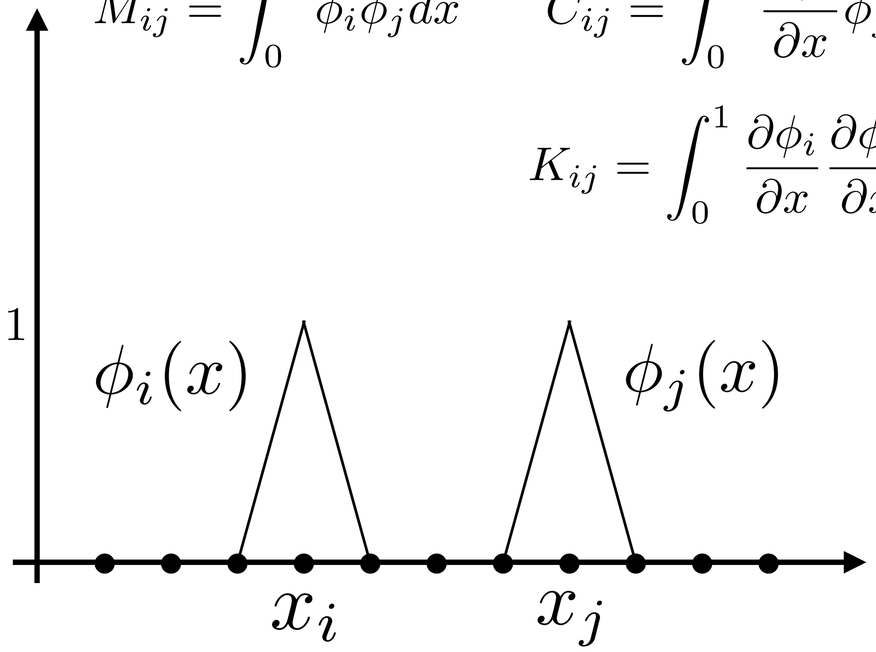
FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, number of elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, C
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$

```
M, K, C = assemble(n_el)
M_solve = spla.splu(M).solve
```

$$M_{ij} = \int_0^1 \phi_i \phi_j dx \qquad C_{ij} = \int_0^1 \frac{\partial \phi_i}{\partial x} \phi_j dx$$

$$K_{ij} = \int_0^1 \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} dx$$

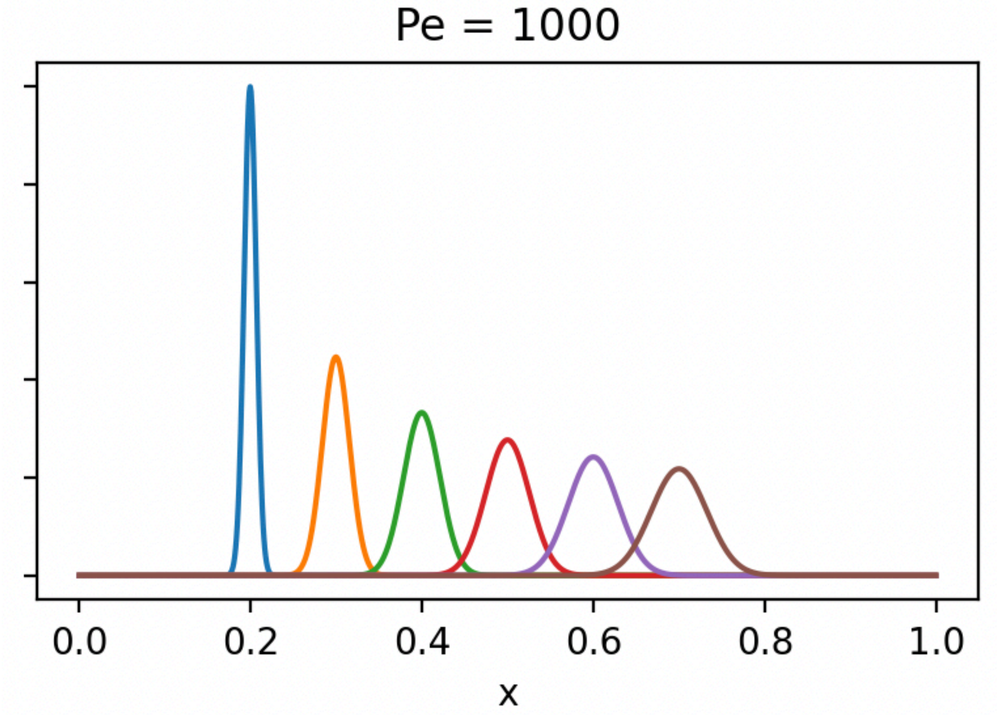


Reduced-Order Modelling

FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, number of elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, C
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$

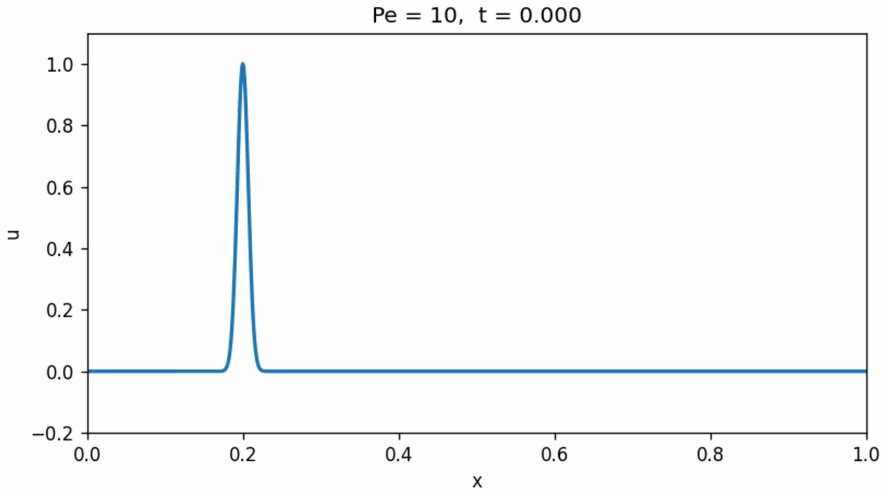
```
u = np.exp(-(((x - x0) / w0) ** 2))
```



Reduced-Order Modelling

FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, number of elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, C
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$



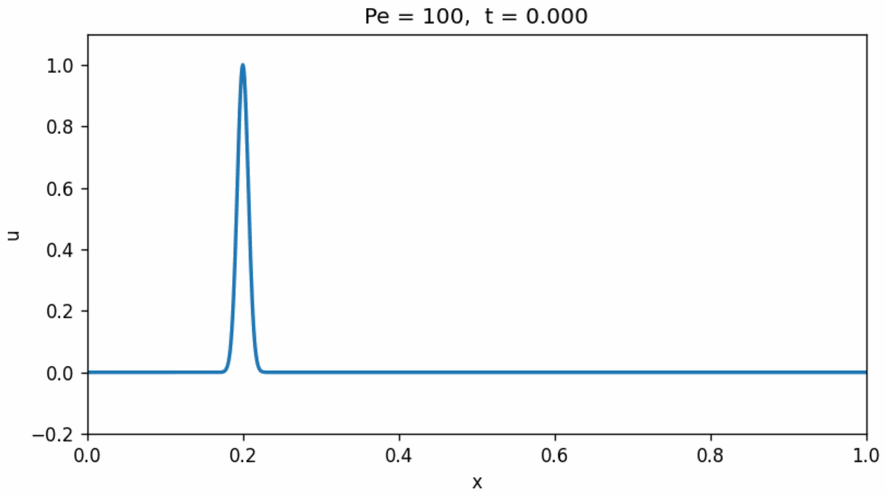

```
for Pe in Pe_values:
    A = K / Pe + C
    u = np.exp(-(((x - x0) / w0) ** 2))
    for k in range(n_steps + 1):
        u = u - dt * M_solve(A @ u)
    S = np.array(S).T

    np.savez(f"snapshots_Pe{Pe}.npz",S=S,t=np.array(col_t),x=x,Pe=Pe)
    print(f"Pe = {Pe}: saved snapshots with shape {S.shape}")
```

Reduced-Order Modelling

FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, number of elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, C
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$



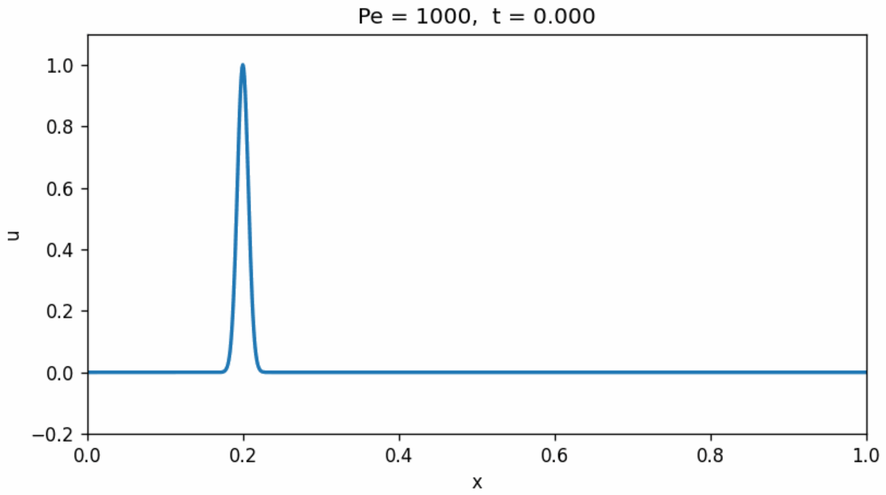

```
for Pe in Pe_values:
    A = K / Pe + C
    u = np.exp(-(((x - x0) / w0) ** 2))
    for k in range(n_steps + 1):
        u = u - dt * M_solve(A @ u)
    S = np.array(S).T

    np.savez(f"snapshots_Pe{Pe}.npz",S=S,t=np.array(col_t),x=x,Pe=Pe)
    print(f"Pe = {Pe}: saved snapshots with shape {S.shape}")
```

Reduced-Order Modelling

FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, number of elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, C
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$




```
for Pe in Pe_values:
    A = K / Pe + C
    u = np.exp(-(((x - x0) / w0) ** 2))
    for k in range(n_steps + 1):
        u = u - dt * M_solve(A @ u)
    S = np.array(S).T

    np.savez(f"snapshots_Pe{Pe}.npz",S=S,t=np.array(col_t),x=x,Pe=Pe)
    print(f"Pe = {Pe}: saved snapshots with shape {S.shape}")
```

Reduced-Order Modelling

FEM for advection diffusion

1. Load FE matrices
2. Load Snapshot matrix
3. Compute the correlation matrix
4. Solve the eigen-value problem
5. Pick the size of the ROM (i.e. N)
6. Get the matrix $\mathbf{Z}$ which represents the change of basis

```
# ------------------------------------------------
# Load data
# ------------------------------------------------

M = sp.load_npz("./M.npz").toarray()
K = sp.load_npz("./K.npz").toarray()
C = sp.load_npz("./C.npz").toarray()

data100 = np.load("./snapshots_Pe10.npz")
data1000 = np.load("./snapshots_Pe1000.npz")

S100 = data100["S"]
S1000 = data1000["S"]

x = data100["x"]
```

Reduced-Order Modelling

FEM for advection diffusion

1. Load FE matrices
2. Load Snapshot matrix
3. Compute the correlation matrix
4. Solve the eigen-value problem
5. Pick the size of the ROM (i.e. N)
6. Get the matrix $\mathbf{Z}$ which represents the change of basis

```
Cpod = S.T @ M @ S
```

Reduced-Order Modelling

FEM for advection diffusion

1. Load FE matrices
2. Load Snapshot matrix
3. Compute the correlation matrix
4. Solve the eigen-value problem
5. Pick the size of the ROM (i.e. N)
6. Get the matrix $\mathbf{Z}$ which represents the change of basis

```
lam, W = la.eigh(Cpod)
```

Reduced-Order Modelling

FEM for advection diffusion

1. Load FE matrices
2. Load Snapshot matrix
3. Compute the correlation matrix
4. Solve the eigen-value problem
5. Pick the size of the ROM (i.e. N)
6. Get the matrix $\mathbf{Z}$ which represents the change of basis

$$E_n = \frac{\sum_{i=1}^{n} \lambda_i}{\sum_{i=1}^{n_{train}} \lambda_i}$$

$$N = \min \{N : \; E_n \geq 1 - \varepsilon\} .$$

```
energy = lam / np.sum(lam)
cumulative_energy = np.cumsum(energy)

N = np.searchsorted(cumulative_energy, energy_target) + 1
```

$$N = \min \left\{ N : \; \frac{\sum_{i=1}^{N} \lambda_i}{\sum_{i=1}^{n_{train}} \lambda_i} \geq 1 - \varepsilon \right\}$$

Reduced-Order Modelling

FEM for advection diffusion

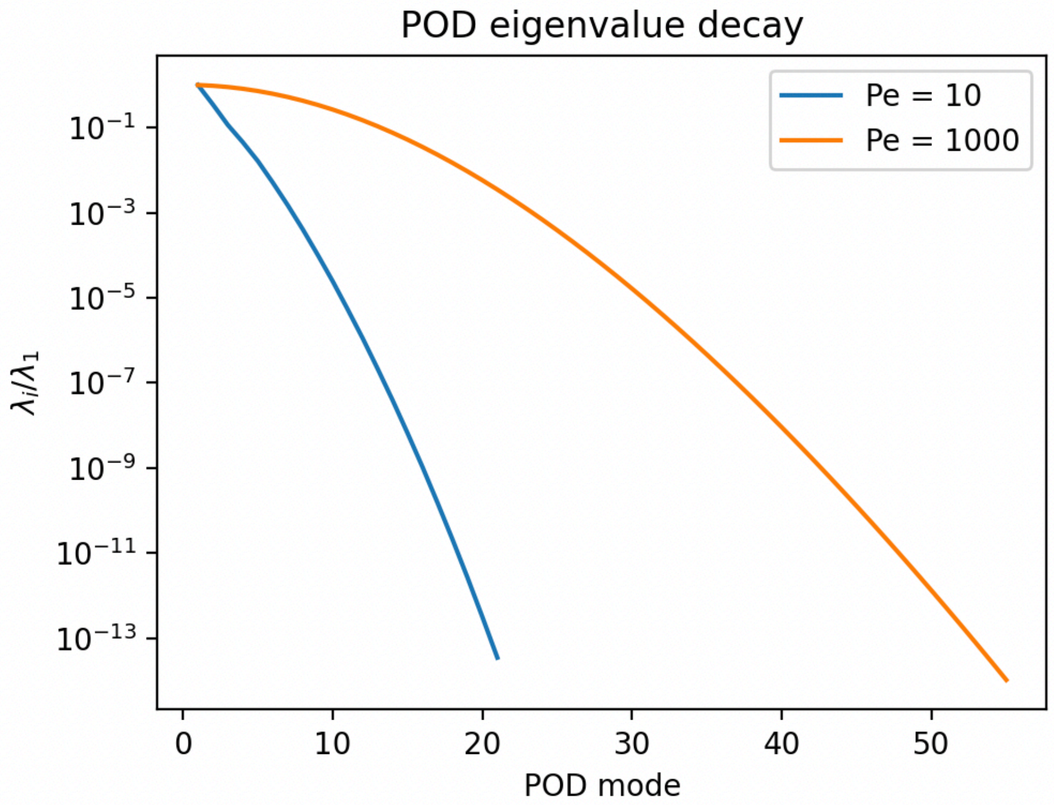
1. Load FE matrices
2. Load Snapshot matrix
3. Compute the correlation matrix
4. Solve the eigen-value problem
5. Pick the size of the ROM (i.e. N)
6. Get the matrix $\mathbf{Z}$ which represents the change of basis

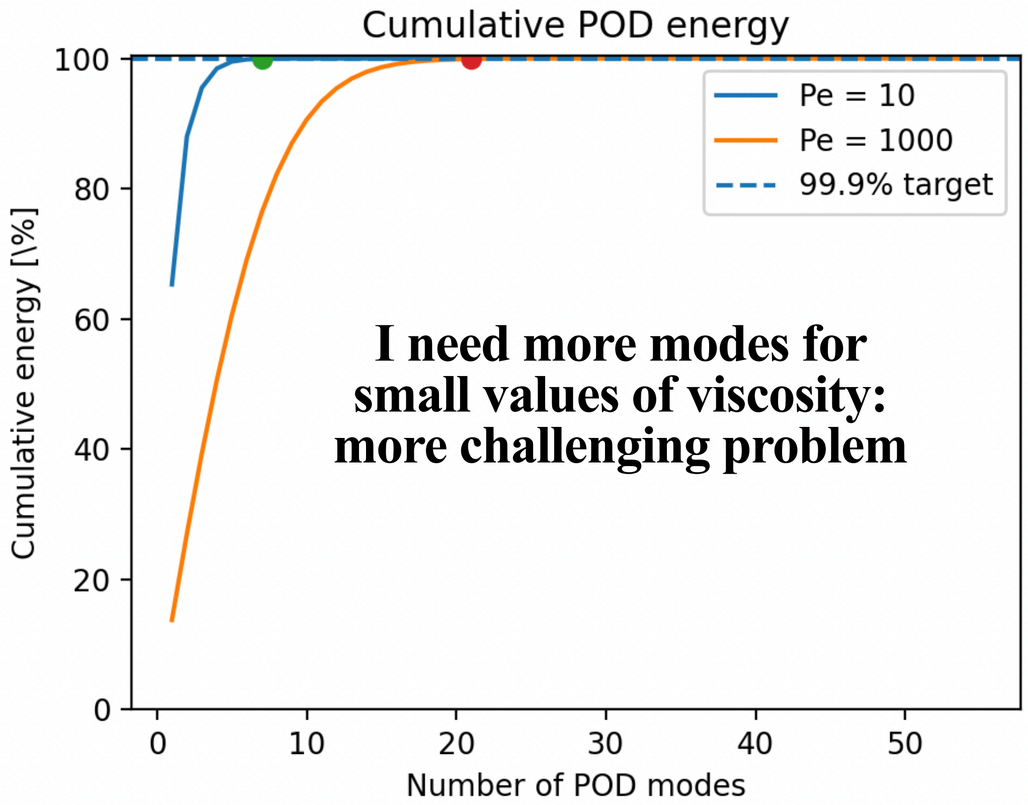
```
Z = S @ W[:, :N] / np.sqrt(lam[:N])
```

Reduced-Order Modelling

FEM for advection diffusion

After you load the matrix of snapshots and solve the associated eigenvalue problem, the first useful information we get is related to the decay of the eigenvalues:

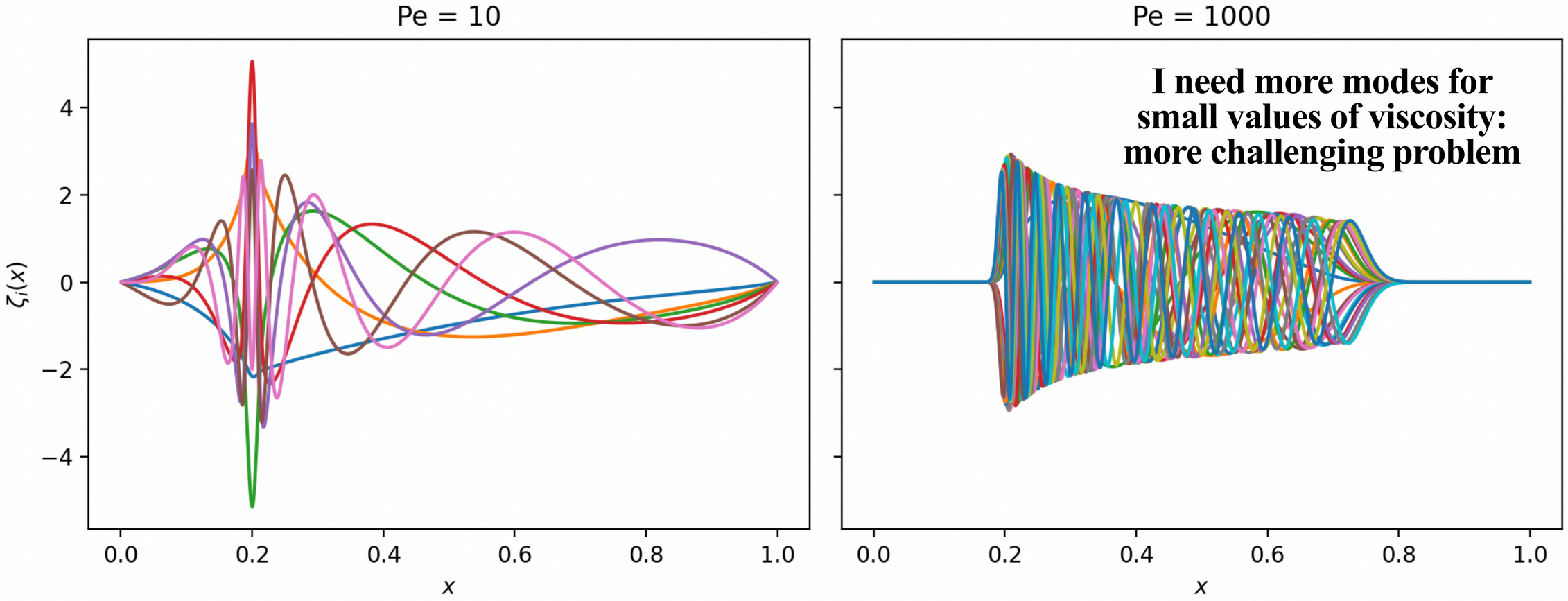




Reduced-Order Modelling

FEM for advection diffusion

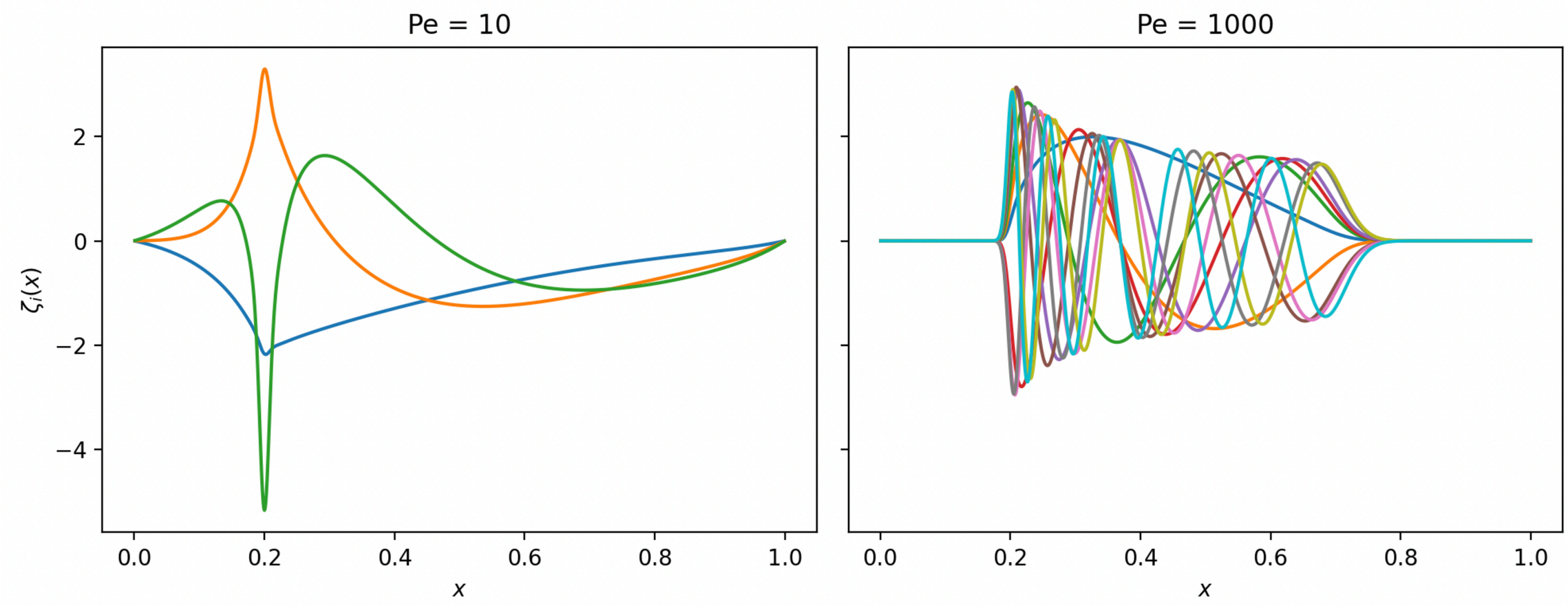
After you load the matrix of snapshots and solve the associated eigenvalue problem, the first useful information we get is related to the decay of the eigenvalues:



Reduced-Order Modelling

FEM for advection diffusion

After you load the matrix of snapshots and solve the associated eigenvalue problem, the first useful information we get is related to the decay of the eigenvalues:



Reduced-Order Modelling

FEM for advection diffusion

Now, we do have our low dimensional representation which is identified by the coefficients of the new basis we picked. We need an equation to evolve those coefficients. In projection based ROMs this come from the projection of the equations themselves.
Instead of solving the FE setting we introduced:

$$\mathbf{M}\frac{\mathbf{u}^{t+1}-\mathbf{u}^{t}}{\Delta t}+\mathbf{C}\mathbf{u}^{t}=-\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}^{t}$$

We solve:

$$\boxed{\mathbf{Z}^{T}\mathbf{M}\mathbf{Z}}\frac{\mathbf{u}_{N}^{t+1}-\mathbf{u}_{N}^{t}}{\Delta t}+\boxed{\mathbf{Z}^{T}\mathbf{C}\mathbf{Z}}\mathbf{u}_{N}^{t}=-\frac{1}{\mathrm{Pe}}\boxed{\mathbf{Z}^{T}\mathbf{K}\mathbf{Z}}\mathbf{u}_{N}^{t}$$

Reduced-Order Modelling

FEM for advection diffusion

$$\boxed{\mathbf{Z}^T\mathbf{M}\mathbf{Z}}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\boxed{\mathbf{Z}^T\mathbf{C}\mathbf{Z}}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\boxed{\mathbf{Z}^T\mathbf{K}\mathbf{Z}}\mathbf{u}_N^t$$

Reduced-Order Modelling

FEM for advection diffusion

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t$$

We first build the reduced operators that we need for the resolution of the low-dimensional system

```
#·-----------------------------------------------¬
#·Reduced·operators¬
#·-----------------------------------------------¬
¬
Kr100·=·V100.T·@·K·@·V100¬
Cr100·=·V100.T·@·C·@·V100¬
Ar100·=·Kr100·/·10·+·Cr100¬
¬
Kr1000·=·V1000.T·@·K·@·V1000¬
Cr1000·=·V1000.T·@·C·@·V1000¬
Ar1000·=·Kr1000·/·1000·+·Cr1000¬
```

Reduced-Order Modelling

FEM for advection diffusion

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t$$

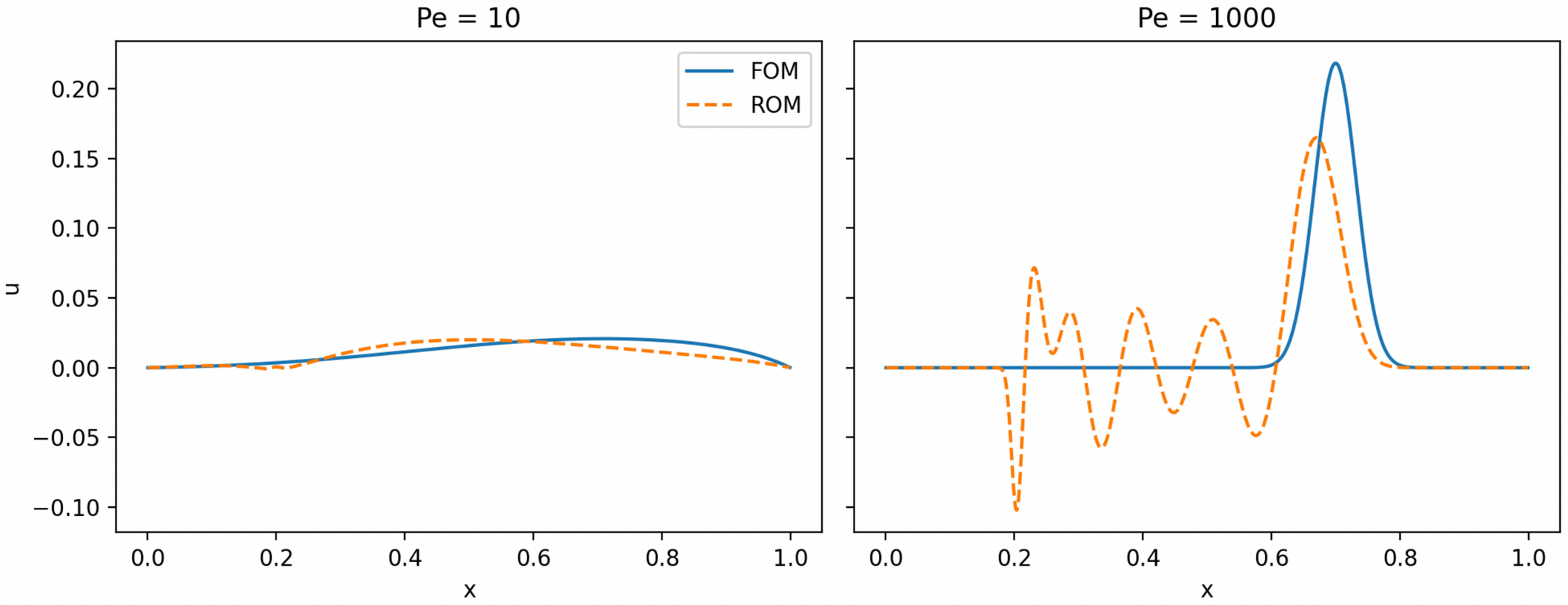
Then we iterate exactly as before
but on the low-dimensional system:

```
def rom(Z,S,Pe):
    A = Z.T @ (K/Pe+C) @ Z
    a = Z.T @ M @ S[:,0]
    start = time.perf_counter()
    for _ in range(n_steps):
        a -= dt*(A@a)
    return Z@a,time.perf_counter()-start
```

Reduced-Order Modelling

FEM for advection diffusion

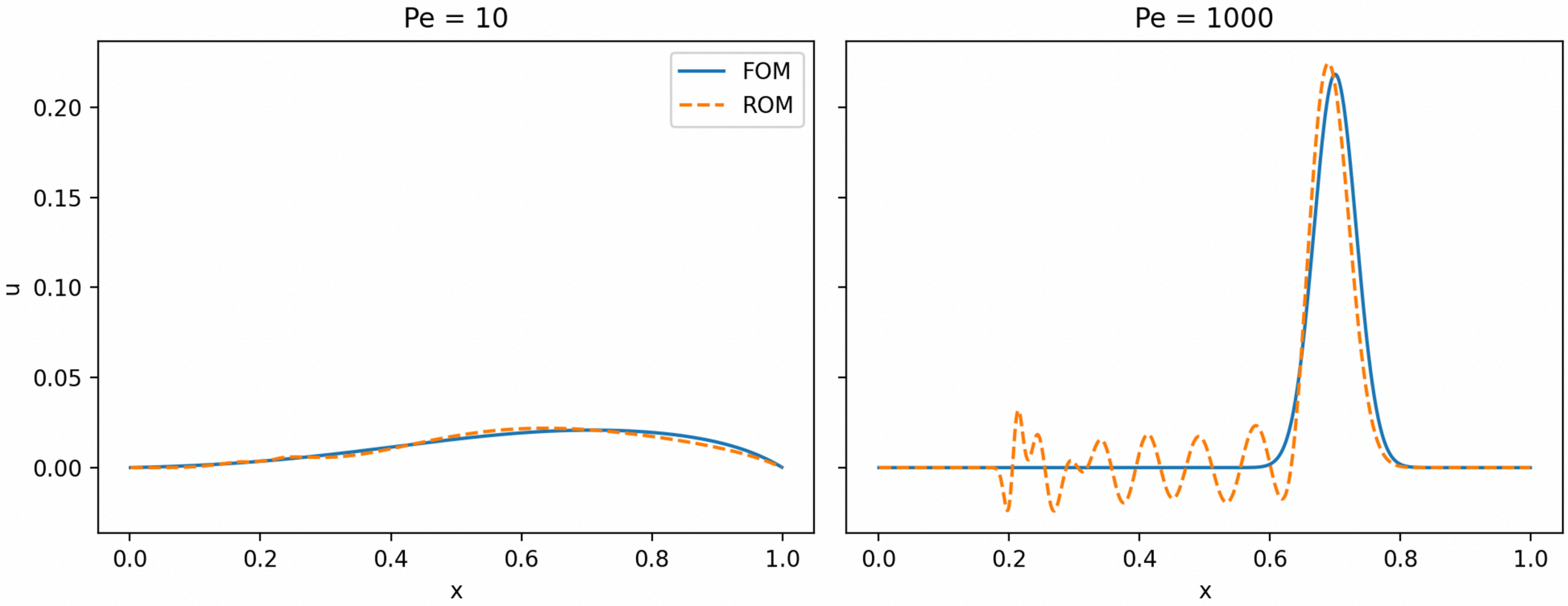
Let’s see how well the ROM performs:



Reduced-Order Modelling

FEM for advection diffusion

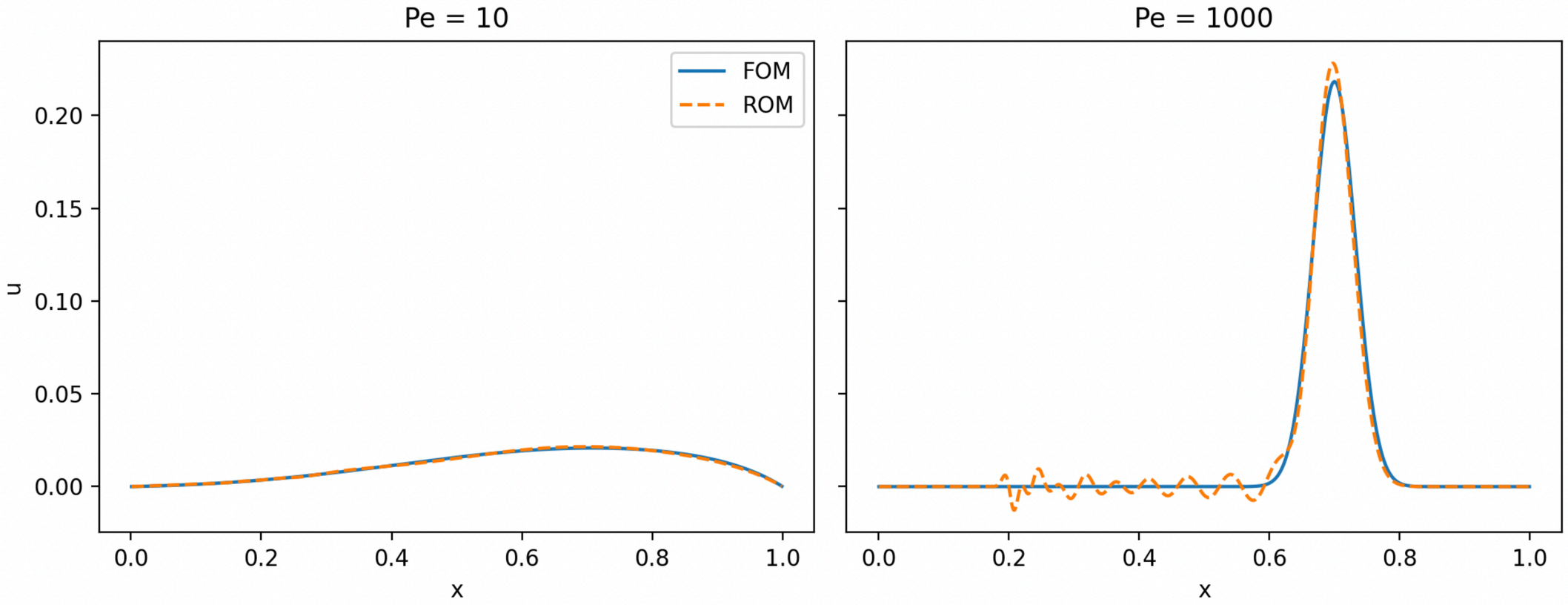
Let’s see how well the ROM performs:



Reduced-Order Modelling

FEM for advection diffusion

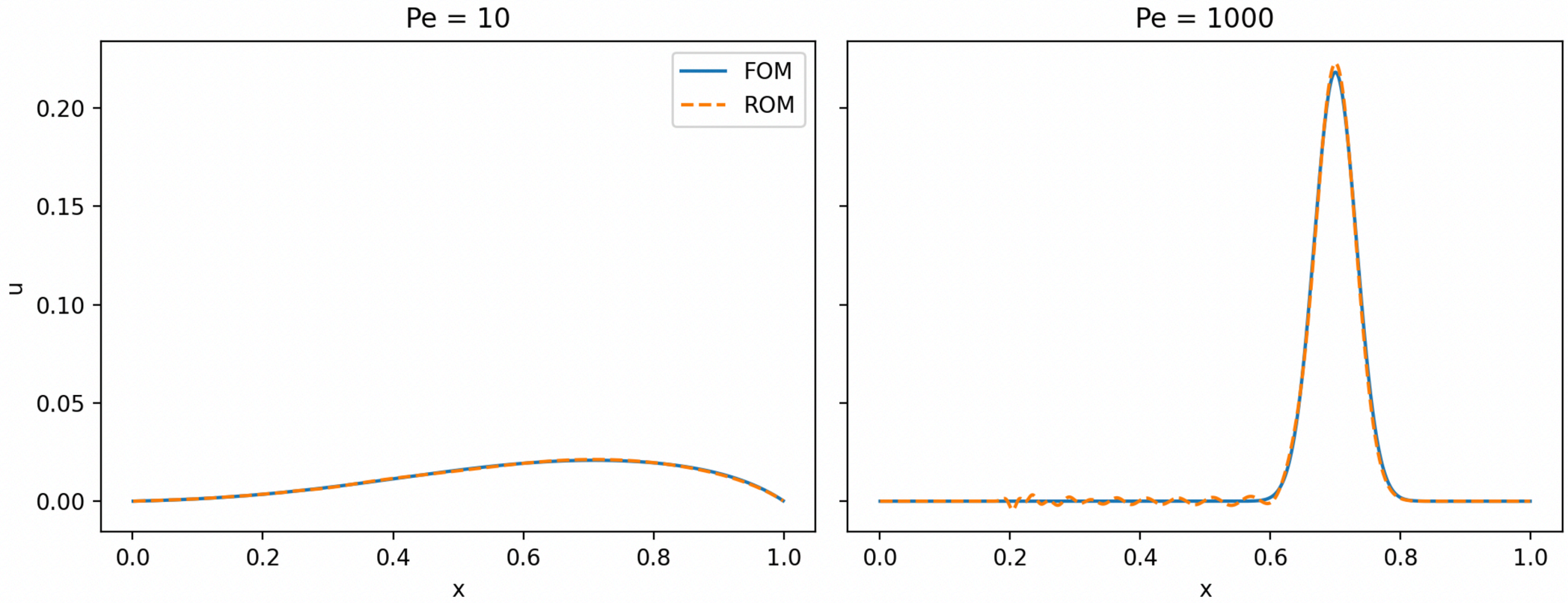
Let’s see how well the ROM performs:



Reduced-Order Modelling

FEM for advection diffusion

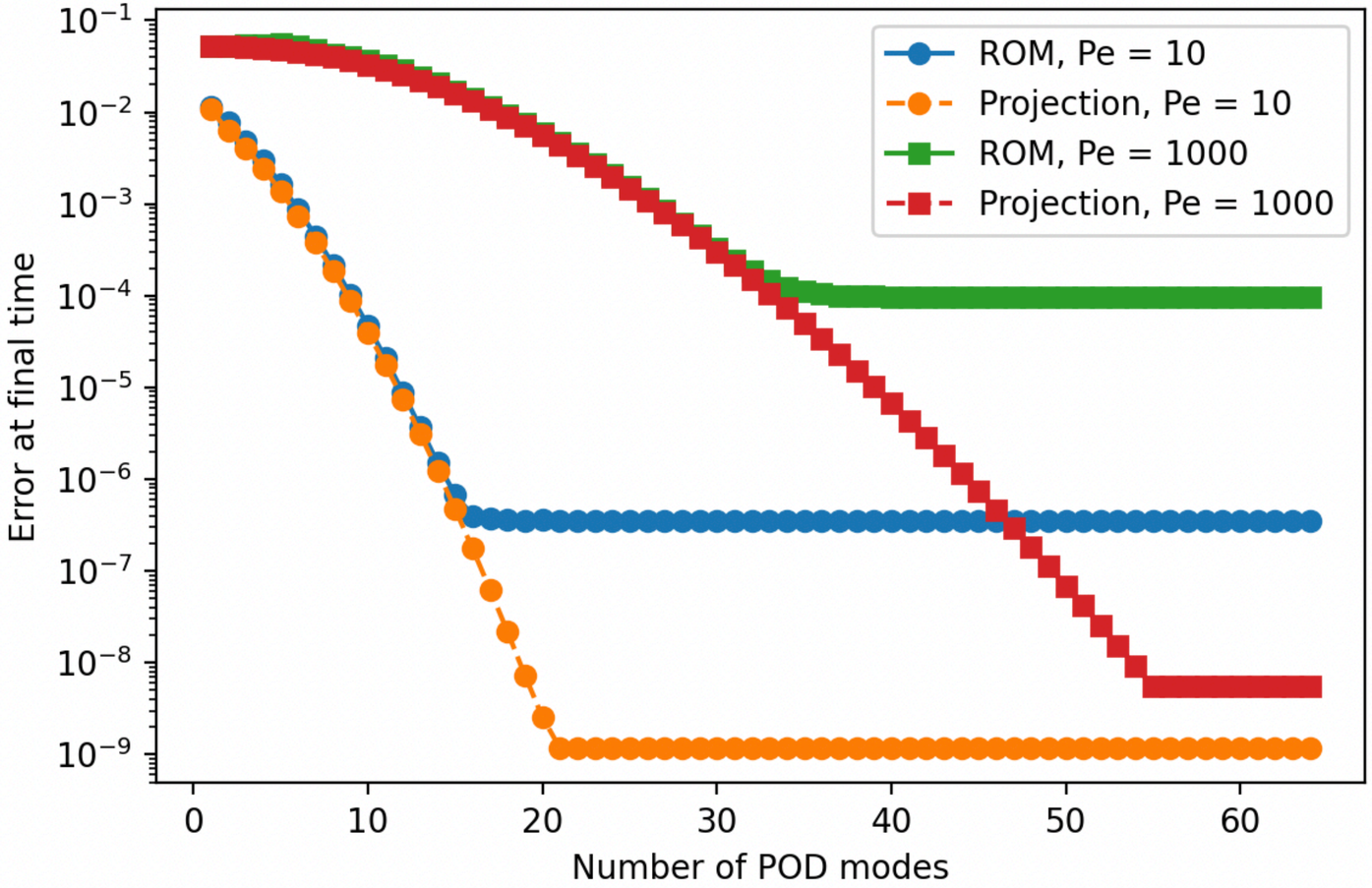
Let’s see how well the ROM performs:



Reduced-Order Modelling

FEM for advection diffusion

Convergence analysis:



Some interpretation of the results:

1. As the number of modes retain increases you generally get smaller errors
2. The error goes down faster for smaller values of Pe (more viscous cases)
3. There are two different types of errors: projection errors and error in the dynamics of the ROM
4. All the errors at some point saturate
    1. Projection error saturates at 1e-8 simply cause I am working in single precision
    2. The ROM saturates because the error of the dynamics is dominant

Reduced-Order Modelling

FEM for advection diffusion

The whole point of ROMs, however, is about speed-ups. So, is the ROM faster than the FOM?

Full-order model (FE)

```
Pe = 10: runtime = 20.539 s
```

Reduced-order model

```
ROM runtime: Pe = 10: 0.921 s
```

So, even when we retain a very large percentage of total energy, we get a speed-up of approximatively 20. We will see that for non-intrusive ROMs we can get much better than this

Reduced-Order Modelling

FEM for advection diffusion

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \frac{1}{\mathrm{Pe}}\frac{\partial^2 u}{\partial x^2}$$

Now, let's make things more complicated and we add a non-linear term

$$\mathbf{M}\frac{\mathbf{u}^{t+1} - \mathbf{u}^t}{\Delta t} + \mathbf{C}(\mathbf{u}^t)\mathbf{u}^t = -\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}^t$$

Now the matrix C depends on the solution itself and needs to be assembled at every time step

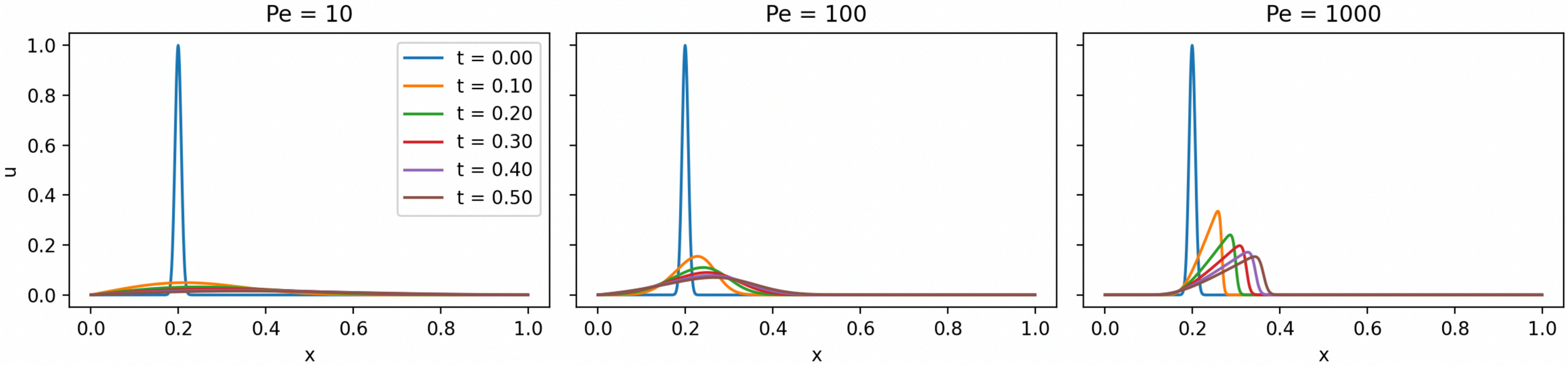
Reduced-Order Modelling

```
Pe = 10: runtime = 178.718 s
Pe = 10: saved snapshots with shape (999, 501)
Pe = 100: runtime = 183.300 s
Pe = 100: saved snapshots with shape (999, 501)
Pe = 1000: runtime = 182.220 s
Pe = 1000: saved snapshots with shape (999, 501)
```

FEM for advection diffusion

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \frac{1}{\mathrm{Pe}}\frac{\partial^2 u}{\partial x^2}$$

Now, let's make things more complicated and we add a non-linear term



Reduced-Order Modelling

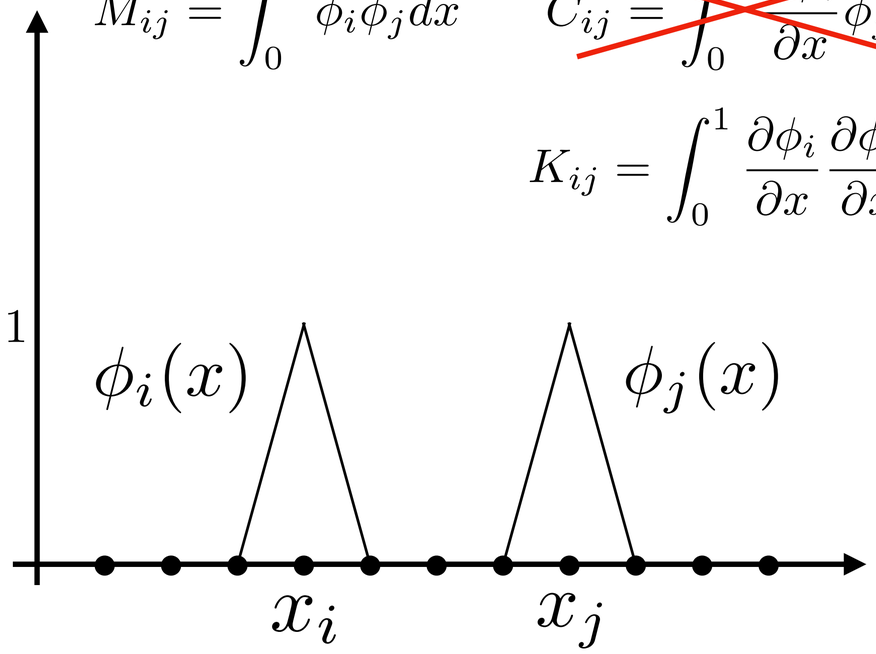
FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, number of elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, ~~C~~
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$

```
M, K, _ = assemble(n_el)
M_solve = spla.splu(M).solve
```

$$M_{ij} = \int_0^1 \phi_i \phi_j dx \qquad C_{ij} = \int_0^1 \frac{\partial \phi_i}{\partial x} \phi_j dx$$

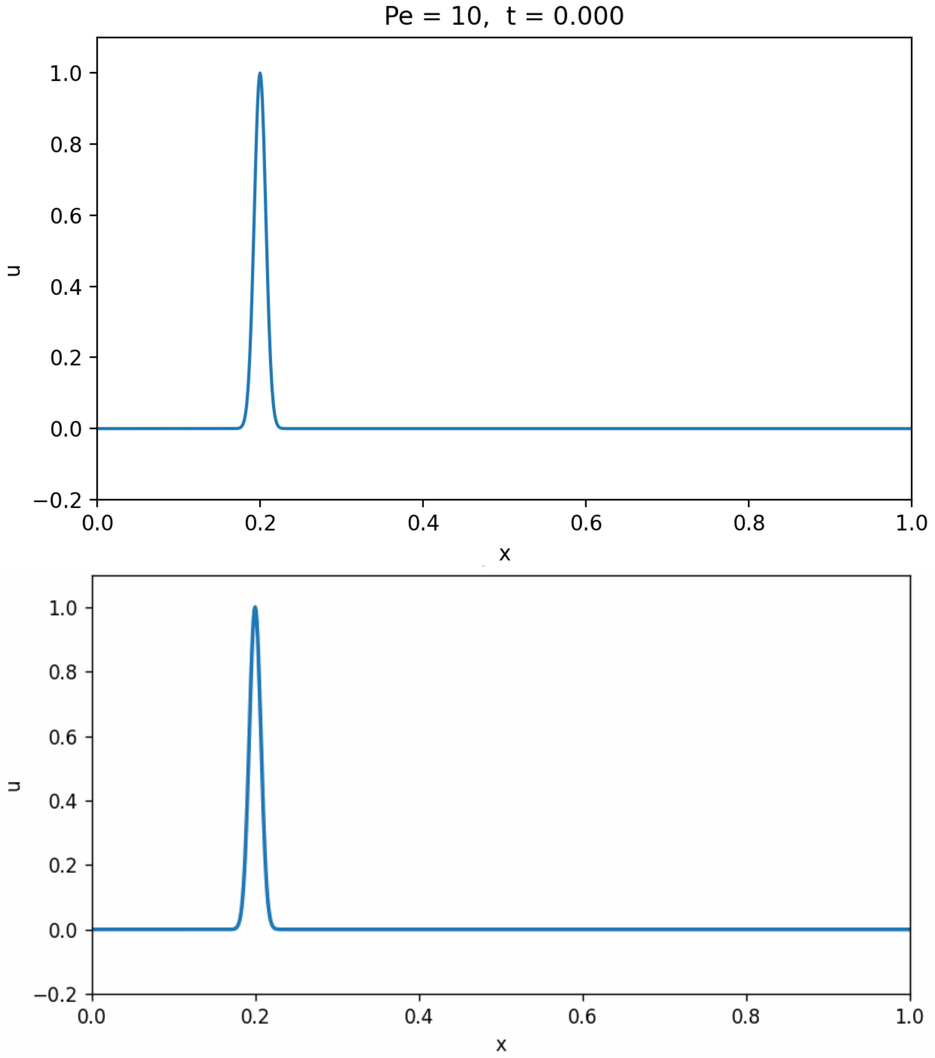
$$K_{ij} = \int_0^1 \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} dx$$



Reduced-Order Modelling

FEM for advection diffusion

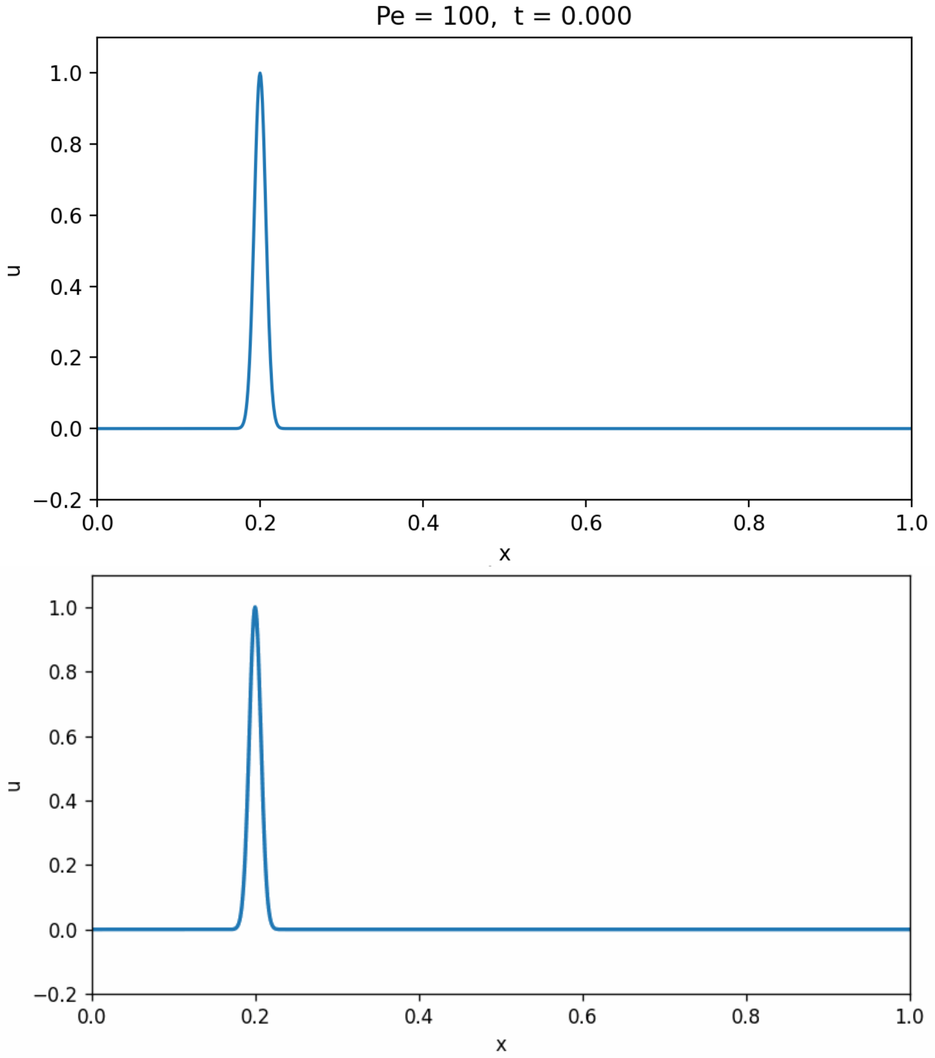
1. Define all the details regarding the discretisation (time step, n elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, ~~C~~
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$



Reduced-Order Modelling

FEM for advection diffusion

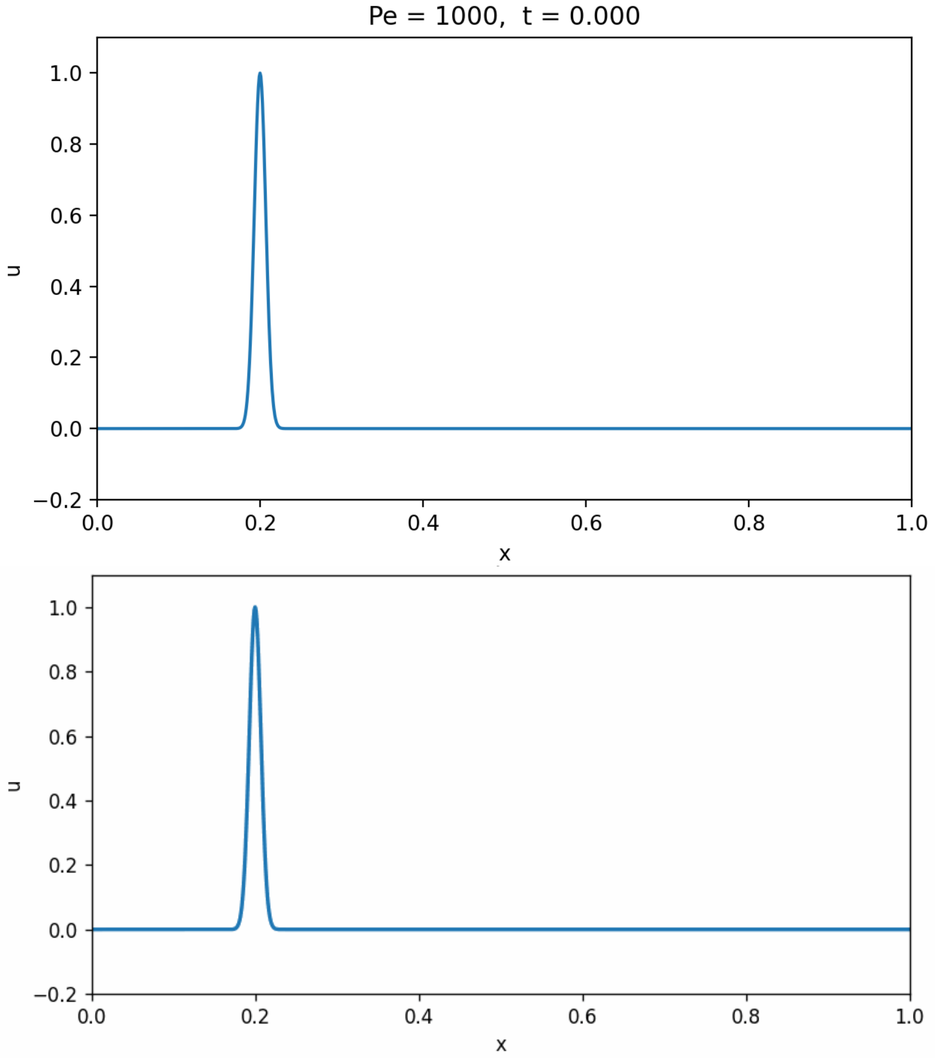
1. Define all the details regarding the discretisation (time step, n elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, ~~C~~
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$



Reduced-Order Modelling

FEM for advection diffusion

1. Define all the details regarding the discretisation (time step, n elements, Peclet number, parametric intial condition)
2. Assemble the FE matrices: M, K, ~~C~~
3. Initialise the mesh and the solution $\mathbf{u}^0$
4. While $t < T$:
   (a) Compute the semi-discrete operator
   $$A = \frac{1}{Pe} K + C$$
   (b) Advance the solution
   $$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n$$
   (c) Update time
   $$t \leftarrow t + \Delta t$$
5. Return the solution $\mathbf{u}(T)$



Reduced-Order Modelling

FEM for advection diffusion

Now, we do have our low dimensional representation which is identified by the coefficients of the new basis we picked. We need an equation to evolve those coefficients. In projection based ROMs this come from the projection of the equations themselves.
Instead of solving the FE setting we introduced:

$$\mathbf{M}\frac{\mathbf{u}^{t+1}-\mathbf{u}^{t}}{\Delta t}+\mathbf{C}(\mathbf{u}^{t})\mathbf{u}^{t}=-\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}^{t}$$

We solve:

$$\mathbf{Z}^{T}\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_{N}^{t+1}-\mathbf{u}_{N}^{t}}{\Delta t}+\mathbf{Z}^{T}\mathbf{C}(\mathbf{Z}\mathbf{u}_{N}^{t})\mathbf{Z}\mathbf{u}_{N}^{t}=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^{T}\mathbf{K}\mathbf{Z}\mathbf{u}_{N}^{t}.$$

Reduced-Order Modelling

FEM for advection diffusion

We solve:

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t.$$

Reduced-Order Modelling

FEM for advection diffusion

We solve:

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t.$$

We can iterate similarly as before but now we have to used the high-dimensional C matrix cause it depends on the high-dimensional solution

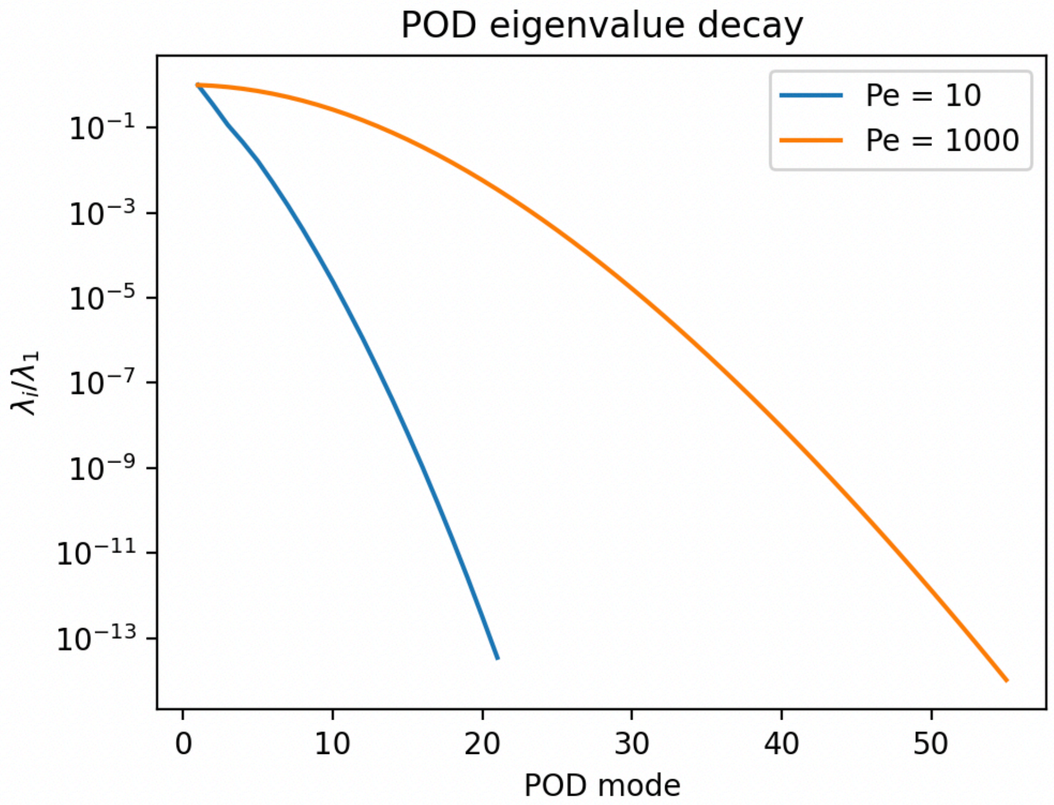
```
def rom(Z, S, Pe):
    a = Z.T @ M @ S[:, 0]
    K_N = Z.T @ K @ Z
    start = time.perf_counter()
    for _ in range(n_steps):
        u = Z @ a
        a -= dt * (K_N @ a / Pe + Z.T @ (C @ (u * u)))

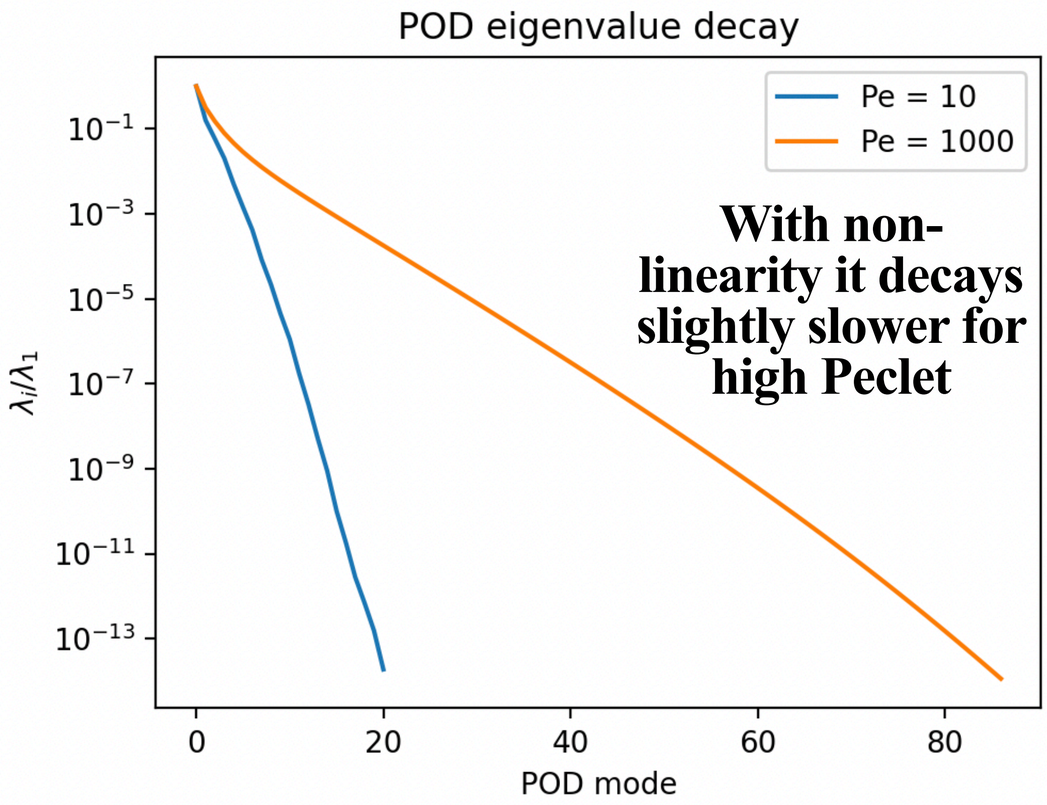
    return Z @ a, time.perf_counter() - start
```

Reduced-Order Modelling

FEM for advection diffusion

After you load the matrix of snapshots and solve the associated eigenvalue problem, the first useful information we get is related to the decay of the eigenvalues:

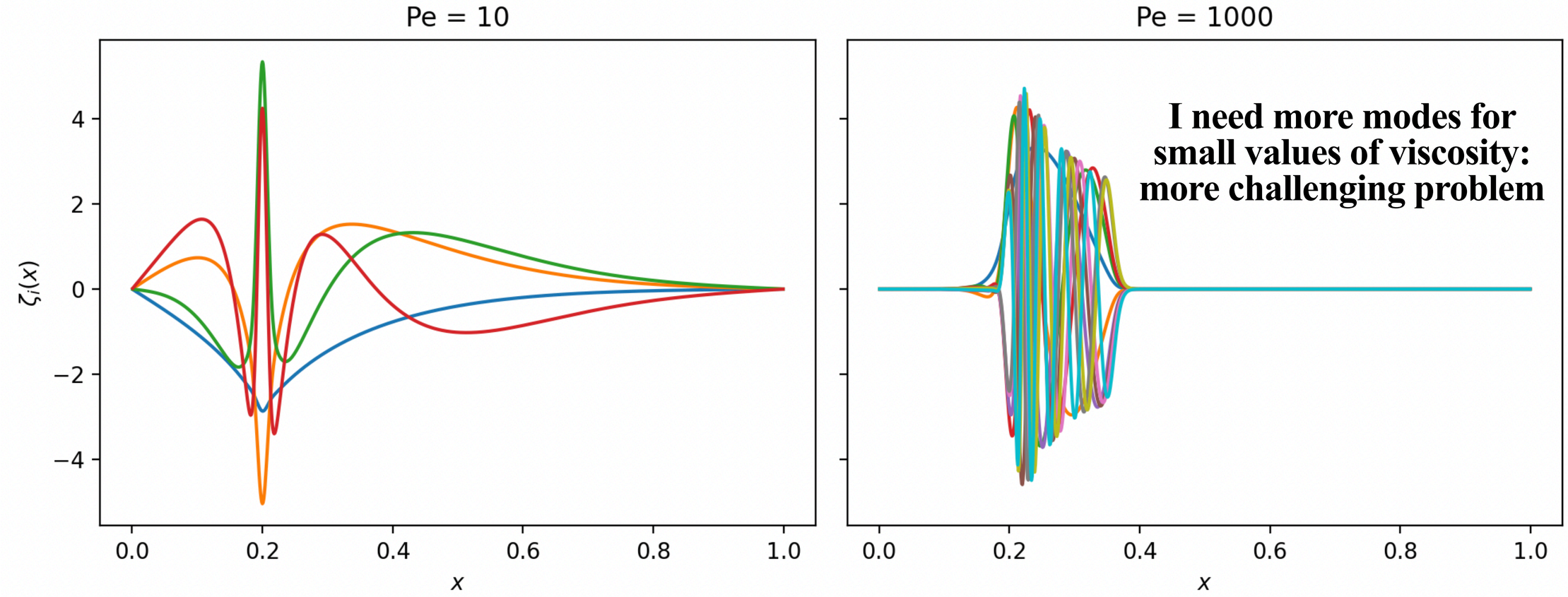




Reduced-Order Modelling

FEM for advection diffusion

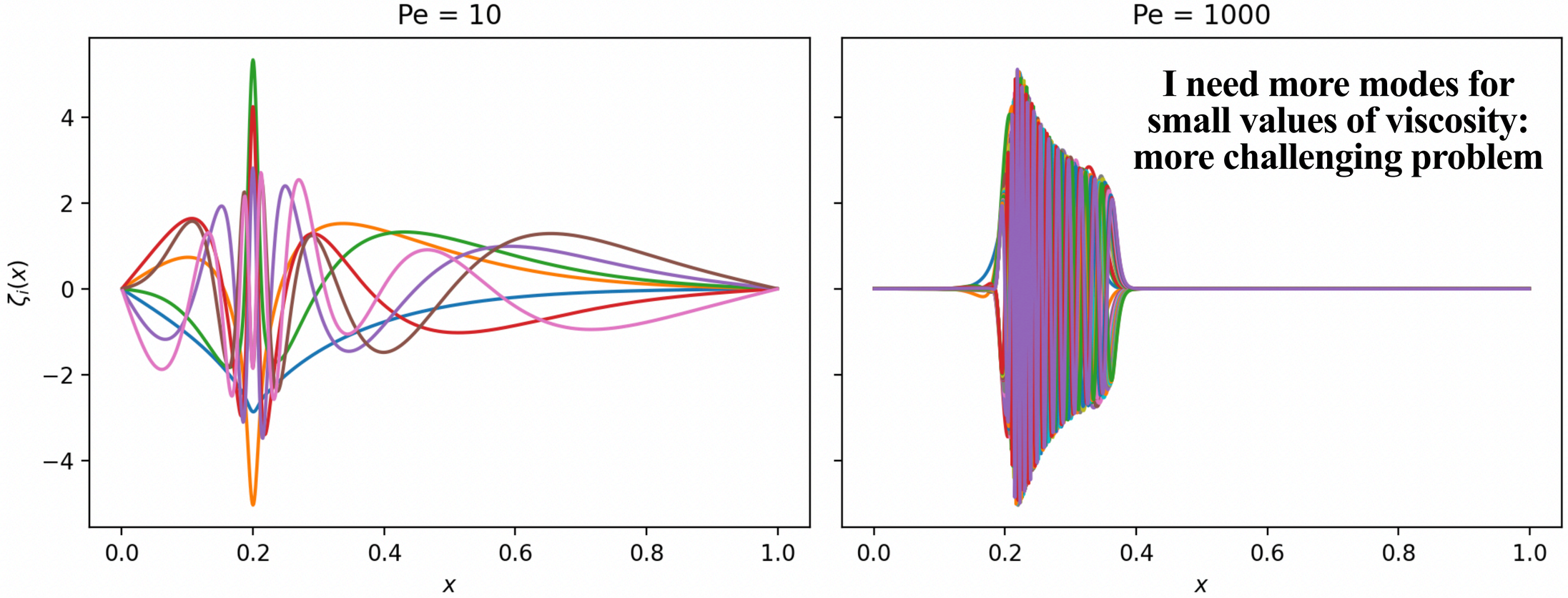
After you load the matrix of snapshots and solve the associated eigenvalue problem, the first useful information we get is related to the decay of the eigenvalues:



Reduced-Order Modelling

FEM for advection diffusion

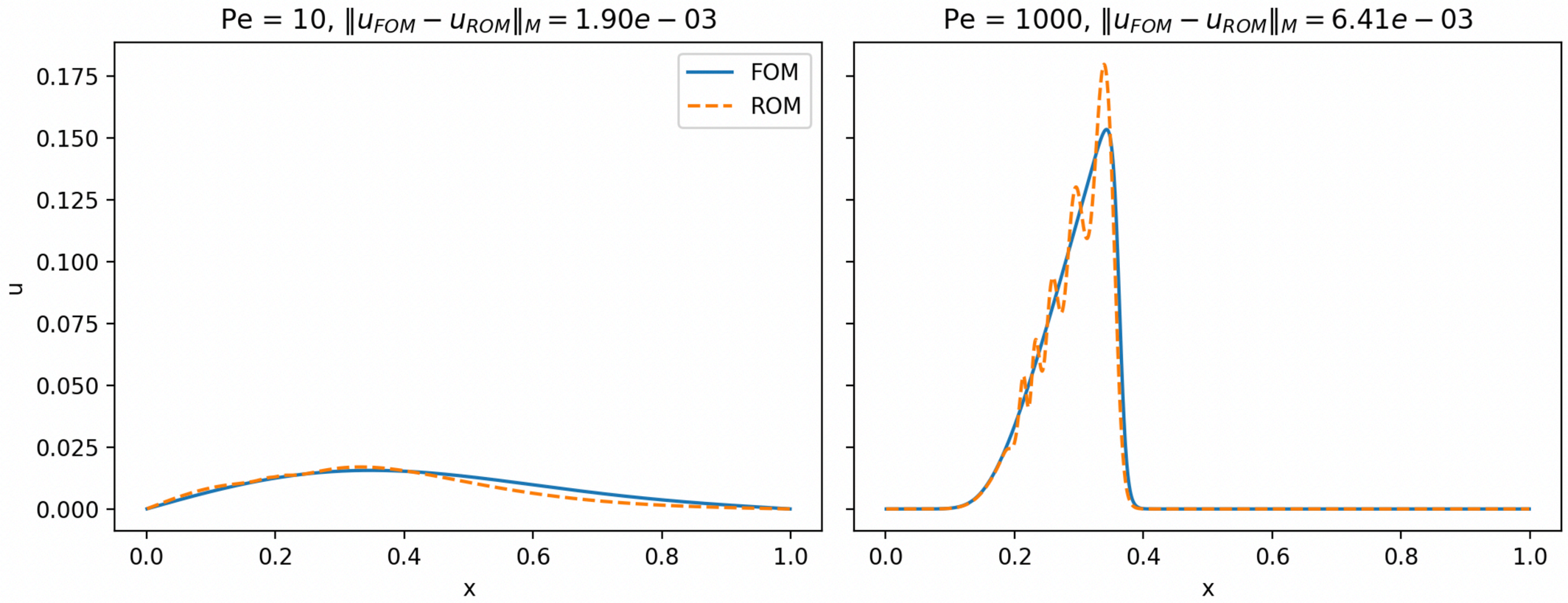
After you load the matrix of snapshots and solve the associated eigenvalue problem, the first useful information we get is related to the decay of the eigenvalues:



Reduced-Order Modelling

FEM for advection diffusion

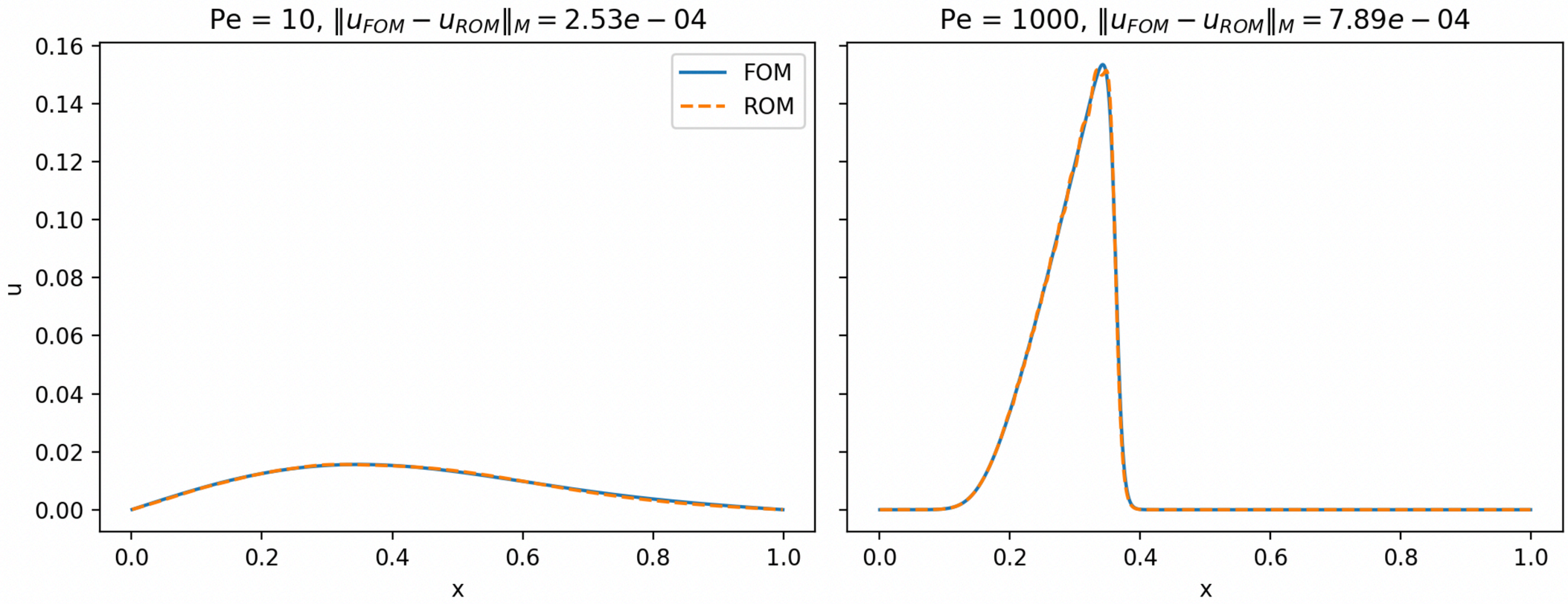
Let's see how well the ROM performs:



Reduced-Order Modelling

FEM for advection diffusion

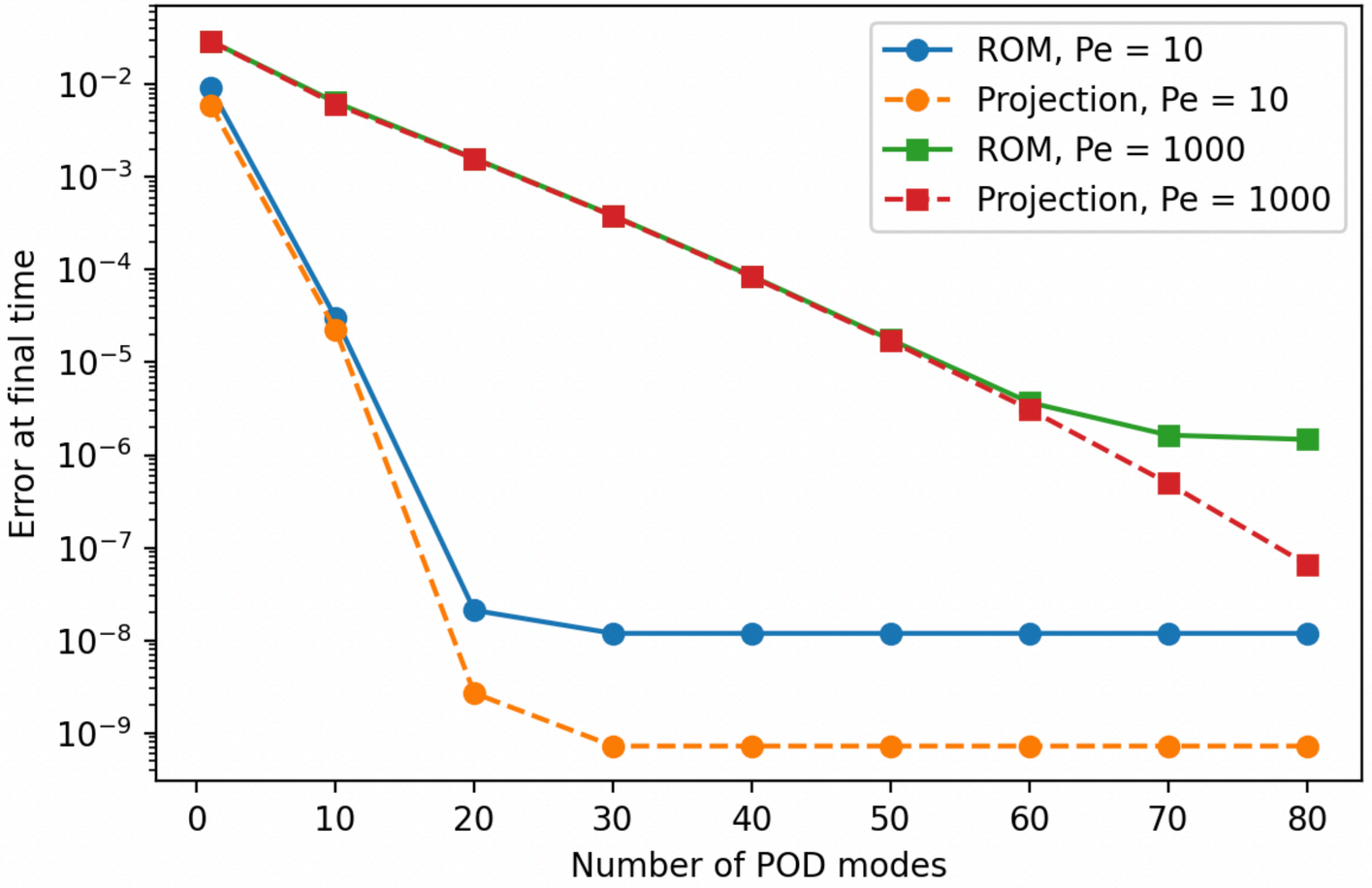
Let's see how well the ROM performs:



Reduced-Order Modelling

FEM for advection diffusion

Convergence analysis:



Some interpretation of the results:

1. As the number of modes retain increases you generally get smaller errors
2. The error goes down faster for smaller values of Pe (more viscous cases)
3. There are two different types of errors: projection errors and error in the dynamics of the ROM
4. All the errors at some point saturate
    1. Projection error saturates at 1e-8 simply cause I am working in single precision
    2. The ROM saturates because the error of the dynamics is dominant

Reduced-Order Modelling

FEM for advection diffusion

The whole point of ROMs, however, is about speed-ups. So, is the ROM faster than the FOM?

Full-order model (FE)

```
Pe = 1000: runtime = 182.220 s
```

Reduced-order model

```
Pe = 1000: 180.001 s
```

So, the ROM takes basically as much time as the FOM!! This is due to the naive way the non-linear term is treated. You end up with operations which scale with the size of the FOM and you lose any advantage.